LEARNING GUIDE · 45–60 MIN

LoRA를 이해하고 MoGe 실험에 적용하기

Low-Rank Adaptation · 저랭크 적응
큰 모델의 가중치를 유지하면서 작은 보정 행렬을 학습하는 방법

기존 지식 작은 보정

$$W_0 + sBA$$

읽고 나면 설명할 수 있는 것

왜 A와 B 두 행렬만 학습하는지, rank와 alpha가 무엇을 조절하는지, 그리고 Teacher의 깊이 결과로 Student를 학습할 때 LoRA가 어떤 역할을 하는지 이해한다.

02–05 기초·수식·직접 계산·학습 과정

06–08 설정 선택·MoGe 구조·메모리

09–12 Teacher–Student·sweep·평가·배포

13–14 복습 문제·답·참고자료

이 자료의 범위

전형준 님의 현재 MoGe 코드와 실험 설정을 예제로 사용한다. 개념 그림과 계산 예제는 학습용이며, 깊이 추정 성능 개선을 보여주는 실험 결과는 아니다.

01 / PREREQUISITES

먼저 알아둘 다섯 가지

신경망 학습은 예측 오차를 줄이도록 숫자 배열인 parameter(파라미터)를 바꾸는 과정이다.

용어	쉬운 설명	MoGe에서의 예
Pretraining · 사전학습	여러 데이터로 기본 능력을 먼저 익힘	이미 학습된 Student 체크포인트
Fine-tuning · 미세조정	새 데이터에 맞게 일부 또는 전체 가중치를 수정	실제 로봇 카메라 영상에 적응
Weight · 가중치	입력 특징을 출력 특징으로 바꾸는 학습 값	attention의 Linear 행렬
Loss / gradient · 손실 / 기울기	오차의 크기 / 값을 바꿀 방향과 민감도	Student 예측과 학습 목표의 차이
Frozen · 고정	해당 파라미터를 optimizer로 갱신하지 않음	LoRA를 붙인 원래 backbone 가중치

Linear layer · 선형층의 약속

$$y = W_0x + b$$

x는 입력 특징, W_0는 기존 가중치, b는 bias(편향), y는 출력이다. 입력 차원이 n, 출력 차원이 m이면 W_0는 m×n이다. 이 자료는 x를 열벡터로 표기한다.

> **코드에서 전치가 보이는 이유**
>
> PyTorch는 배치 입력을 행 단위로 저장하므로 Linear는 XW_0^T+b로 계산한다. 수학 표기와 배열 배치 방식이 달라 보일 뿐 같은 연산이다. [4]

PEFT · 적은 파라미터만 학습하는 방법

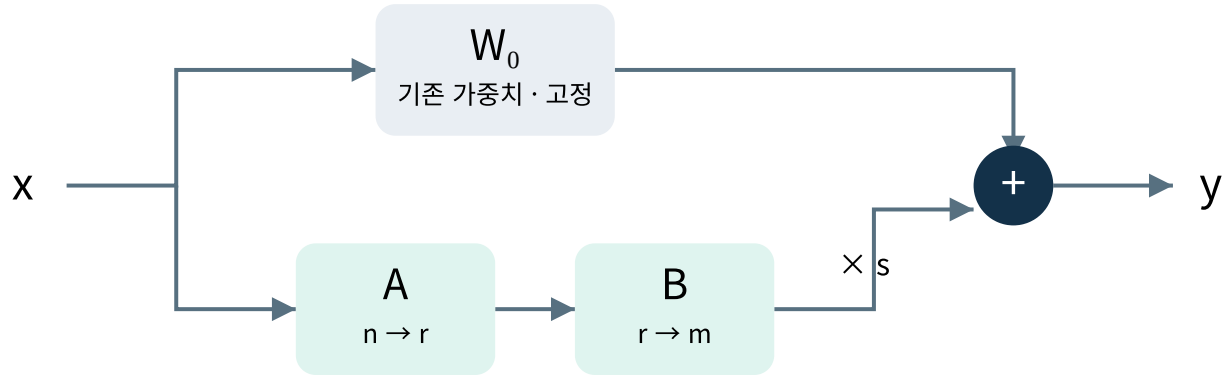
Parameter-Efficient Fine-Tuning은 학습하는 파라미터를 줄이는 방법들의 범주다. LoRA는 그중 하나다. 모델의 모든 가중치를 직접 수정하는 full fine-tuning과 구분한다. [2]

02 / CORE IDEA

기존 출력에 작은 보정을 더한다

LoRA는 원래 가중치 W_0를 고정하고, 그 변화량 ΔW를 두 작은 행렬의 곱으로 표현한다. [1]

$$y = W_0x + b + sB(Ax) \qquad s = \alpha / r$$



회색: 고정된 원래 경로 / 초록: 학습하는 보정 경로. 그림에서는 bias를 생략했다.

기호	크기	역할
W_0	m×n	기존 변환
A	r×n	입력을 r개 중간 성분으로 변환
B	m×r	중간 성분을 출력 공간으로 변환
s = α/r	스칼라	보정 출력에 곱하는 배율

Low rank · 변화량의 표현 범위를 제한

ΔW=sBA의 rank는 최대 r이다. W_0 자체가 저랭크일 필요는 없다. A와 B는 보통 학습으로 찾으며, 매 단계마다 원래 W_0를 SVD로 분해하는 방식은 아니다. [1]

03 / WORKED EXAMPLE

작은 숫자로 직접 계산해 보기

입력 2차원, 출력 2차원, rank 1, s=1인 학습용 예제다. bias는 0으로 둔다.

기존 가중치

$$W_0 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

보정 행렬

$$A = \begin{bmatrix} 1 & 2 \end{bmatrix}, \quad B = \begin{bmatrix} 0.1 \\ 0.2 \end{bmatrix}$$

입력

$$x = \begin{bmatrix} 3 \\ 4 \end{bmatrix}$$

1. 원래 출력

$$W_0x = [3, 4]^T$$

2. 작은 보정 경로

$$Ax = 1\times3 + 2\times4 = 11$$

$$B(Ax) = [1.1, 2.2]^T$$

3. 두 결과를 합치기

$$y = [3, 4]^T + [1.1, 2.2]^T = [4.1, 6.2]^T$$

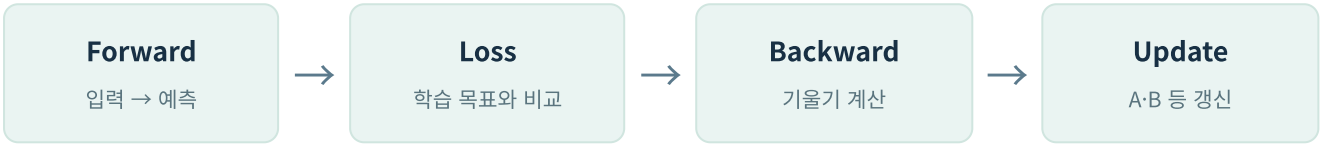
이때 BA=[[0.1, 0.2], [0.2, 0.4]]다. 두 번째 행은 첫 번째 행의 2배이므로 서로 독립적인 행 방향은 하나다. 이것이 rank 1의 한 예다.

> **파라미터 절감은 큰 행렬에서 확인한다**
>
> 이 2×2 예제는 W가 4개, A+B도 4개라 절감이 없다. 384×384 행렬과 rank 4라면 기존 147,456개 대신 4×384+384×4=3,072개를 학습한다. 해당 행렬 기준 약 2.08%다. bias는 제외한 계산이다.

04 / TRAINING

처음에는 기존 모델과 같게 시작한다

현재 구현은 A를 난수로, B를 0으로 초기화한다. 따라서 시작할 때 BA=0이고 기존 출력이 유지된다. [P1]



왜 A와 B를 모두 0으로 두지 않을까?

g=∂L/∂y라 두면, 한 입력에 대한 보정 경로의 기울기는 아래와 같다. 현재 예제에서는 가중치 감쇠 같은 optimizer의 추가 효과를 제외한다.

$$\partial L/\partial B = s \cdot g(Ax)^T$$
$$\partial L/\partial A = s \cdot B^T g\, x^T$$

A와 B가 모두 0이면 두 기울기가 모두 0이 되어 보정 경로가 학습을 시작하지 못한다. A는 난수, B는 0이면 첫 단계에서 B가 먼저 바뀔 수 있고, 이후 A에도 손실 기울기가 흐른다.

Frozen과 no_grad는 다르다

가중치를 고정해도 앞쪽의 학습 파라미터까지 기울기를 전달해야 하면 연산 그래프가 필요하다. LoRA가 들어간 전체 Student backbone을 `torch.no_grad()`로 감싸면 adapter 학습도 끊긴다. Teacher의 오프라인 추론에는 기울기 기록이 필요 없다. [5]

> **현재 코드에서 확인할 항목**
>
> 기존 encoder 파라미터는 고정, lora_A와 lora_B는 학습 가능이어야 한다. Neck·heads도 별도로 학습한다. 저장 후 고정 가중치 불변, adapter 변화, NaN/Inf 유무를 확인한다. [P1,P2]

05 / HYPERPARAMETERS

rank·alpha·학습률은 역할이 다르다

설정을 바꿀 때는 “표현 가능한 변화”와 “업데이트 크기”를 나누어 생각한다.

설정	무엇을 바꾸나	현재 후보
Rank r · 중간 차원	ΔW의 최대 rank와 adapter 파라미터 수	4 / 8
Alpha α · 스케일 설정	보정 출력 배율 s=α/r	8 / 16 → 둘 다 s=2
Learning rate · 학습률	optimizer가 A·B를 갱신하는 크기	1e-5
Target modules · 적용 층	어느 가중치에 LoRA를 붙일지	마지막 4블록 qkv·proj
Dropout · 임의 누락	학습 중 보정 경로 입력 등을 일부 제거	현재 미사용

Rank가 두 배면 성능도 두 배일까?

학습할 수 있는 변화의 범위는 넓어지지만, 실제 품질은 데이터·학습률·적용 위치에 달려 있다. Rank 8이 rank 4보다 낫다는 결론은 평가 후에만 내린다. [2]

α/r를 같게 하면 무엇이 통제될까?

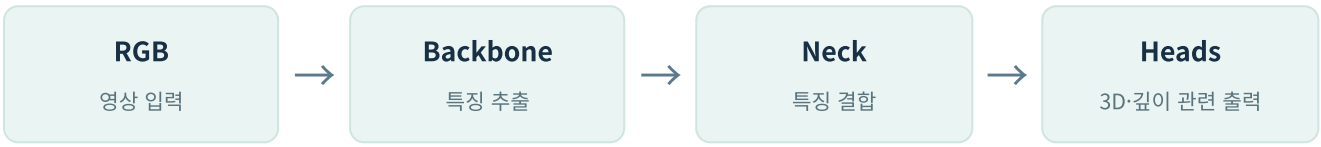
현재 r=4, α=8과 r=8, α=16은 출력 배율을 2로 맞춘다. 다만 A·B의 크기와 최적화 경로가 달라 두 조건의 실제 업데이트 크기까지 같아지는 것은 아니다.

> **이번 sweep을 읽는 방법**
>
> Rank 4와 8은 같은 LoRA 학습률로 비교한다. 일반 학습의 backbone LR은 5e-7이고 LoRA는 1e-5다. 따라서 일반 학습과의 비교는 두 학습 설정의 실용적 비교이며, 동일 학습률에서 LoRA 효과만 분리한 실험은 아니다. [P2]

06 / VISION TRANSFORMER

MoGe에서는 어디에 붙였을까?

현재 Student의 DINOv2 backbone(특징 추출부) 마지막 4개 블록에서 attention의 qkv와 proj에 적용한다. [P1,P2]



앞쪽 backbone 블록 **고정**

마지막 4개 블록

qkv + LoRA → Attention → proj + LoRA

기존 qkv·proj, norm, MLP는 고정

Neck·heads **직접 학습**

Q·K·V는 무엇인가?

Query(조회 벡터)와 Key(비교 벡터)로 어떤 위치의 정보를 참고할지 계산하고, Value(전달할 특징)를 섞는다. proj는 섞인 결과를 다음 특징 공간으로 바꾸는 선형층이다. 여기서는 Q·K·V가 하나의 qkv 행렬에 합쳐져 있어 세 부분 전체를 보정한다.

Rank 4 adapter는 총 몇 개의 숫자를 학습할까?

현재 Small backbone의 특징 차원 d=384를 사용하면 qkv는 1152×384, proj는 384×384이다.

$$\text{블록당: } r(384+1152) + r(384+384) = 2304r$$

$$\text{4블록: } 9216r \rightarrow r=4\text{: } 36{,}864 \;/\; r=8\text{: } 73{,}728$$

모든 학습 파라미터가 adapter는 아니다

위 계산은 LoRA 부분만 센 값이다. 이번 실험은 neck·heads도 학습하므로 전체 학습 파라미터는 약 1,279만 개 또는 1,283만 개다.

구현·배치 위치: [P1,P2] · 파라미터 수: [P3]

07 / MEMORY

학습량 감소와 GPU 메모리 감소

GPU에는 가중치 외에도 기울기, optimizer 상태, activation(중간 계산값), 임시 버퍼가 들어간다.

메모리 구성	LoRA를 쓰면
원래 모델 가중치	추론 계산에 필요하므로 계속 존재
기울기·optimizer 상태	고정된 파라미터 부분은 줄일 수 있음
Activation · 중간 계산값	adapter까지 역전파할 경로에 필요할 수 있음
Neck·heads 및 연산 버퍼	현재처럼 직접 학습하면 계속 필요

우리 코드의 짧은 동작 확인 결과

조건	전체 학습 파라미터	Peak allocated
일반 last4	19,852,328	1.424 GiB
LoRA rank 4	12,788,264	1.400 GiB
LoRA rank 8	12,825,128	1.401 GiB

[P3] 로컬 Student 10,000회 best, 실제 혼합 데이터, 배치 1×누적 32, 각 1회 업데이트. Validation 없음. 서버의 18,500회 best 및 본 학습 배치 8 결과가 아니다.

Rank 4의 전체 학습 파라미터는 약 35.6% 줄었지만 이 측정의 peak allocated는 약 1.7% 줄었다. 학습 파라미터 수의 비율을 GPU 메모리 절감률로 그대로 해석할 수 없다.

> **LoRA와 QLoRA**
>
> QLoRA는 양자화된 고정 모델과 LoRA를 결합해 기본 가중치 저장 비용도 줄이는 접근이다. 현재 MoGe 구현은 4-bit 양자화를 사용하지 않는 일반 LoRA다. QLoRA의 LLM 실험 수치를 이 모델에 그대로 적용할 수 없다. [7]

메모리 원리: [1,5] · 관측: [P3] · QLoRA: [7]

08 / DISTILLATION

Teacher 학습과 LoRA는 함께 쓴다

Teacher는 학습 목표를 제공하고, LoRA는 Student에서 어떤 파라미터를 갱신할지 정한다.



Knowledge distillation · 지식 증류

큰 모델의 예측을 학습 신호로 사용해 다른 모델에 능력을 옮기는 발상이다. [6] 현재 코드는 분류 확률의 temperature나 KL loss를 사용하는 방식이 아니라, 미리 저장한 metric depth(실제 거리 단위 깊이)를 이용하는 오프라인 pseudo-label 학습이다. [P4]

현재 데이터와 손실

CARLA의 GT(기준 정답)를 75%, 실제 Teacher pseudo를 25% 확률로 샘플링한다. Pseudo 쪽 손실 항의 기존 가중치에는 0.25를 추가로 곱한다. 이 두 비율은 서로 다른 설정이다. [P2]

$$\text{개념적 기대 손실} \approx 0.75\ \mathbb{E}[L_{\text{CARLA}}] + 0.25 \times 0.25\ \mathbb{E}[L_{\text{pseudo}}]$$

이 식은 샘플링과 가중치 관계를 설명한다. 실제 손실은 여러 기하·스케일 항으로 구성되며, 기울기 기여 비율이 위 계수와 같다는 뜻은 아니다.

> **Teacher가 좋아 보였다는 관찰의 활용**
>
> 실제 영상에 적응시킬 근거가 되지만 Teacher 예측도 오차를 포함한다. 현재 pseudo에는 normal·mask 직접 지도 손실을 제외하고, 유효한 깊이 픽셀을 사용한다. 별도 촬영 세션에서 개선을 확인해야 한다. [P2,P4]

09 / EXPERIMENT DESIGN

세 조건을 같은 출발점에서 비교

Sweep(조건별 반복 실행)은 이전 조건의 결과를 이어받지 않고 같은 Student에서 독립적으로 시작한다.

조건	Backbone에서 학습	LR	α/r
direct_last4	마지막 4블록 기존 파라미터	5e-7	해당 없음
lora_r4	마지막 4블록 A·B	1e-5	8/4=2
lora_r8	마지막 4블록 A·B	1e-5	16/8=2

공통 조건

Student는 `best_flip_mild_t400_total18500.pt`, 입력 400 tokens(특징 토큰), 3,000회 업데이트, 유효 배치 32다. 데이터·증강·seed·CARLA 평가 조건을 맞추며 neck·heads 학습은 모두 유지한다. [P2]

실행 순서

```
cd ~/MoGe
source .venv-moge/bin/activate

# 설정 생성만
python run_script/run_v4l2_lora_sweep.py --dry-run

# 조건별 최대 10회 업데이트, validation 생략
python run_script/run_v4l2_lora_sweep.py --smoke

# 세 조건 각각 3,000회
python run_script/run_v4l2_lora_sweep.py
```

기본 pseudo 경로는 `/home/coder/dataset/v4l2_pseudo_v1`이다. 메모리가 부족하면 `--batch-size 4`로 세 조건을 동일하게 조정한다. 누적 8회로 유효 배치 32를 유지한다.

> **추가하면 좋은 대조 조건**
>
> 추가 학습 전 Student를 평가에 포함한다. 실제 pseudo의 효과까지 분리하려면 같은 횟수의 CARLA-only 추가 학습도 별도 비교한다. 현재 3조건 sweep에는 CARLA-only가 자동 포함되지는 않는다.

10 / EVALUATION

무엇이 개선됐는지 나누어 평가

Teacher와 더 비슷해졌다는 사실만으로 실제 거리 정확도가 좋아졌다고 판단할 수는 없다.

확인할 질문	평가 방법
기존 CARLA 성능이 유지되는가?	고정 validation의 REL↓, δ1↑ 비교
실제 장면의 형태가 좋아졌는가?	별도 촬영 세션에서 같은 장면을 나란히 비교
실제 거리 단위가 맞는가?	거리 기준이 있는 장면이나 독립 GT에서 오차 확인
장면 간 일관성이 있는가?	카메라·거리·조명·얇은 구조별로 나눠 확인

지표의 뜻

$$\mathrm{REL} = \mathrm{mean}\left(|\hat{d} - d| / d \right)$$

$$\delta 1 = \mathrm{mean}\left(\max(\hat{d}/d, d/\hat{d}) < 1.25 \right)$$

d는 유효한 기준 깊이, $\hat{d}$는 예측 깊이다. 예를 들어 실제 2m를 2.2m로 예측하면 해당 픽셀의 상대 오차는 0.1이다. 평가 mask·범위·정렬 방식은 모든 조건에서 같아야 한다.

시각 비교 패널 구성

RGB 입력 → **Teacher** 참고 → **기존 Student** 기준 → **학습 Student** 후보

모든 depth 그림의 거리 범위와 color scale(색-거리 대응)을 고정한다. 프레임마다 색 범위를 자동 조정하면 거리 오차가 가려질 수 있다. 경계·바닥·근거리 장애물과 실제 m 단위를 함께 확인한다.

> **현재 데이터의 한계**
>
> 현재 252장의 pseudo 데이터는 모두 학습용이다. 같은 영상의 인접 프레임을 평가용으로 나누면 장면 누수가 생길 수 있으므로 별도 촬영 세션을 확보한다. Summary의 CARLA best만으로 실제 환경 최적 모델을 결정하지 않는다. [P4]

11 / INFERENCE & DEPLOYMENT

학습이 끝나면 병합해서 사용

선형 보정이므로 $W^*=W_0+sBA$로 합치면 일반 Linear와 같은 구조로 추론할 수 있다. [1,3]

$$W^* = W_0 + (\alpha/r)BA$$

$$W^*x + b = W_0x + b + (\alpha/r)B(Ax)$$

병합 전과 후

상태	저장 내용·특징
현재 LoRA 체크포인트	기본 가중치 + A·B + 학습된 neck·heads + 설정
일반 모델로 병합	변경된 W* + 학습된 neck·heads, 별도 adapter 없음

일반적인 adapter-only 저장은 작지만 정확히 같은 base 모델이 따로 필요하다. 현재 코드는 완전한 모델 파일을 저장하므로 “LoRA 체크포인트는 항상 작다”는 설명이 적용되지 않는다. [3,P1]

```
python tools/merge_moge2_lora.py \
  --checkpoint workspace/v4l2_lora_sweep/RUN/lora_r4/checkpoint/best.pt \
  --output pretrained/student400_v4l2_lora_r4_merged.pt
```

RUN은 실제 생성된 실행 폴더명으로 바꾼다. 출력 파일은 새 이름을 사용한다.

실행 후 확인할 것

같은 RGB와 같은 전처리로 병합 전후의 points·normal·mask·metric scale을 비교한다. 부동소수점 연산 순서 때문에 미세한 차이는 있을 수 있다. 최종 배포 환경에서도 입력 크기·토큰 수·거리 단위를 확인한다.

> **이번 구현에서 확인한 범위**
>
> Rank 4 동작 확인 체크포인트를 병합하고 FP32 출력의 수치 일치를 확인했다. 이는 구현 검증이며 깊이 성능 개선의 증거는 아니다. 별도 저장한 merged 파일에는 optimizer 재개 상태가 없다. [P1,P3]

12 / SELF CHECK

다섯 문제로 이해 확인하기

답을 보기 전에 각 질문을 한두 문장으로 설명해 본다.

01 **Rank 8이면 원래 W_0도 rank 8일까?**
어느 행렬의 rank에 제한이 걸리는지 적어 보자.

02 **왜 A와 B를 모두 0으로 초기화하지 않을까?**
5쪽의 두 미분 식에 A=0, B=0을 대입해 보자.

03 **512×512 층에 rank 8을 붙이면?**
기존 행렬과 A+B의 파라미터 수, 학습 비율을 구하자. Bias는 제외한다.

04 **Teacher가 고정이면 Student도 학습이 안 될까?**
학습 목표를 생성하는 모델과 gradient를 받는 모델을 구분해 보자.

05 **LoRA rank 8의 train loss만 더 낮다면?**
어떤 평가가 있어야 더 나은 모델이라고 말할 수 있을까?

짧은 답

01 $\Delta W = sBA$의 rank가 최대 8이다. W_0의 rank 제한은 없다.
02 두 손실 기울기가 모두 0이 되기 때문이다.
03 262,144개 대 8,192개, 비율 3.125%.
04 고정 Teacher가 목표를 제공하고 Student의 A·B 및 neck·heads는 갱신된다.
05 독립 실제 세션의 품질·거리 오차, CARLA 성능 유지와 자원 비용을 비교해야 한다.

13 / REFERENCES

원문과 코드를 이어서 읽기

일반 원리는 원 논문과 공식 문서에서, 실제 적용 범위는 현재 저장소 코드에서 확인한다.

[1] Hu et al. · LoRA (2021; ICLR 2022)
4.1절의 재매개화, 4.2절의 Transformer 적용부터 읽는다.
arxiv.org/abs/2106.09685

[2] Hugging Face PEFT · LoRA 공식 문서
rank, alpha, target modules와 초기화 설정을 확인한다.
huggingface.co/docs/peft/v0.21.0/package_reference/lora

[3] Microsoft · LoRA 공식 구현
가중치 고정, adapter 저장, 병합 및 fused qkv 관련 설명.
github.com/microsoft/LoRA

[4] PyTorch · Linear / [5] Autograd mechanics
docs.pytorch.org/docs/stable/generated/torch.nn.Linear.html
docs.pytorch.org/docs/stable/notes/autograd.html

[6] Hinton et al. · Distilling the Knowledge in a Neural Network (2015)
arxiv.org/abs/1503.02531

[7] Dettmers et al. · QLoRA (2023)
arxiv.org/abs/2305.14314

현재 저장소의 근거 파일

[P1] moge/model/modules/lora.py · tools/merge_moge2_lora.py
[P2] run_script/run_v4l2_distillation.py
run_script/run_v4l2_lora_sweep.py
configs/experiments/v4l2_lora_sweep.json
[P3] workspace/v4l2_lora_sweep/integration_smoke/summary.json
동일 폴더의 merge_verification.json
[P4] docs/v4l2_distillation.md · tools/v4l2_distillation.py

자료 확인: 2026-11-05. 현재 MoGe 구현은 자체 LoRA 모듈을 사용하며 PEFT 라이브러리에 의존하지 않는다. 링크된 문서는 개념 참고용이다.